

CIS 6261: Trustworthy Machine Learning

Final Project Report [Option 1]: Adversarial Training as a Dual Defense: Improving Both Robustness and Privacy

Ruobin Chen
(*Point of Contact*)
rchen6@ufl.edu

Anup Sedhain
anupsedhain@ufl.edu

Donghao Li
d.li2@ufl.edu

Ibrahim Mohammed
i.mohammed@ufl.edu

December 4, 2025

1 Introduction

Adversarial machine learning is a field of study that is rapidly growing in importance and complexity. As machine learning becomes further integrated into the daily lives of people. It is more open to opportunities for adversaries to harm humanity, like how computer viruses and malware have evolved in the past few decades. As adversarial examples, membership inference, model extraction, and other forms of attacks evolve, so do the countermeasures against them. It becomes a perpetual game of cat and mouse for new, adaptive attacks and defenses to counter each other. The continuous implementation of defense is necessary to address and prevent new threats that can sabotage systems, violate privacy, and proprietary technology. In this project, we practice this flow by developing attacks and defenses on adversarial examples and membership inference with a vision model.

The project has two parts. In part 1, we defend a target model against membership inference attacks (MIAs) and adversarial example attacks (AEAs) in the input domain of CIFAR-10 images. We are given two MIAs and one AEA to evaluate the undefended model and observe that the model performs rather poorly against them. Our goal is to create a defense without retraining, meaning that it is focused on preprocessing inputs and postprocessing outputs, that is more robust against these attacks. To further evaluate robustness, we implement additional MIAs and AEAs to test the defense against potential weak points. We propose a prediction-time randomized defense that adds input noise, rotation/translation, temperature scaling, and output noise. We implement loss-based, label-based, and shadow model-based, MIAs and gradient-based and optimization-based AEAs to test the defense's ability to be robust against different adversarial approaches while maintaining good accuracy.

In part 2 of the project, we shift our focus from defending a pre-trained model to training a robust model from scratch. We explore the effectiveness of adversarial training as a defense mechanism. This involves training a ResNet-18 model on the CIFAR-100 dataset, incorporating adversarial examples generated by a Projected Gradient Descent (PGD) attack directly into the training process. We build a model that is inherently more resilient to adversarial attacks while also evaluating its privacy leakage using the membership inference attacks developed in Part 1. This approach allows us to investigate the trade-offs between robustness, privacy, and model accuracy in a more complex and realistic setting.

2 Background & Related Work

A membership inference attack [6] is defined by the question of "given a machine learning model and a record, determine whether this record was used as part of the model's training dataset or not". The discovery of this ability to infer on a private training dataset revealed a risk to using sensitive data, such as medical records, to train a model. In the first membership inference attack implementation on ML models by Shokri et al. [6], a shadow model approach was used to learn how the model prediction reacts differently to data in and out of its training set. This attacks often work under specific assumptions like knowing the target model's architecture, loss distribution, and output vectors. Since then, simpler attacks have been designed to maintain or improve attack efficacy while working until stricter constraints. [5] shows that the shadow model approach can be simplified to have lower cost and more relaxed assumptions. [8] shows that the cross entropy loss of a model's predictions can be used to conduct membership inference attacks. [2] shows that multiple engineered queries can inference membership even if the model reveals nothing except the predicted class.

Adversarial attacks are small perturbations to input data that can cause machine learning models to produce incorrect outputs, even when the changes are nearly invisible to the human eye. This phenomenon was explored in depth by Goodfellow et al. [3], who introduced the Fast Gradient Sign Method (FGSM). FGSM showed that by taking a single step in the direction of the input gradient, one can create an adversarial example that can fool a wide range of models. This exposed a vulnerability to the linear behavior of models with high-dimensional input spaces.

Following FGSM, stronger iterative methods were proposed to exploit this weakness further. Madry et al. [4] framed adversarial robustness from an optimization perspective. Their Projected Gradient Descent (PGD) is an iterative attack that repeatedly applies small gradient updates while keeping the final perturbation within a fixed range. PGD proved itself as a strong enough attack to use as a standard reference for evaluating the robustness of a model.

Carlini and Wagner [1] took a different approach and chose to search for minimal perturbations in different norms (L_0, L_2, L_∞). The CW attack is known for being able to evade naive or obfuscated defenses, highlighting that some strategies fail under more advanced methods. Their work also emphasized the need for rigorous attack evaluations when proposing robustness claims.

Additional studies have broadened the landscape of adversarial research. Properties such as "transferability" have arisen, where adversarial examples crafted for one model can fool another. Further work is being done on "black-box" attacks that have limited knowledge on the classifier parameters or access to the training dataset. The attacks discussed in this report are all "white-box" attacks in which the adversarial attack has knowledge of the classifier parameters.

3 Approach: Dataset(s) & Technique(s)

3.1 Dataset

In part 1, we use the CIFAR-10 dataset, which consists of 32x32 pixel images split between 10 classes. The target model uses the ResNet-18 architecture and is trained with 20,000 images. Additional images are used for validating/testing prediction accuracy and membership inference performance. In part 2, we instead use the CIFAR-100 dataset, which increases the problem complexity to 100 classes. We will use the same ResNet-18 architecture from Part 1 to allow for a consistent comparison of defense effectiveness.

3.2 Defense Mechanism

This defense mechanism has a primary purpose: maintain high performance on defending attacks while keeping the models as accurate as possible. We deploy a purely inference-time randomized defense. For the ℓ -th forward of input $x \in \mathbb{R}^{C \times H \times W}$:

$$z_\ell(x) = \frac{1}{T_\ell} f(g_{\theta_\ell, \Delta_\ell}(x + \eta_\ell)) + \xi_\ell, \quad T_\ell = T(1 + \delta u_\ell),$$

with $\eta_\ell \sim \mathcal{N}(0, \sigma^2 I)$, $g_{\theta, \Delta}$ (bilinear rotate $\theta_\ell \in [-\Theta, \Theta]$ & horizontal shift $\Delta_\ell \in \{-S, \dots, S\}$), $\xi_\ell \sim \mathcal{N}(0, \gamma^2 I)$, and $u_\ell \sim \mathcal{U}[-1, 1]$. We average L randomized forwards (EOT):

$$\bar{z}(x) = \frac{1}{L} \sum_{\ell=1}^L z_\ell(x), \quad \hat{y} = \arg \max_c \text{softmax}(\bar{z}(x))_c.$$

3.2.1 Hyperparameters and Roles

- σ (**input Gaussian noise**): smooths decisions; weakens threshold/consistency MIAs; small clean-acc trade-off.
- Θ (**max rotation, deg**): breaks rotation-consistency assumptions; slight clean-acc cost.
- S (**max horizontal shift, px**): breaks shift(+rotation) consistency; keep small to avoid boundary artifacts.
- T (**temperature**): softens logits, reduces confidence/loss separability; too large may blunt decisions.
- δ (**temperature jitter**): $T_\ell = T(1 + \delta u_\ell)$; specifically destabilizes per-example loss (loss-based MIAs).
- γ (**tiny logit noise**): blurs exploitable score ties; often negligible impact; set $\gamma = 0$ for AE-only eval.
- L (**EOT runs**): logit averaging over L randomizations; improves AE stability; runtime $\propto L$.

3.2.2 Implementation Notes

We use bilinear interpolation with zero fill for geometry; perturbation magnitudes are intentionally small to preserve usability. Privacy-oriented tuning increases $(\sigma, T, \delta, \gamma)$ moderately; robustness-oriented tuning increases (σ, L) and slightly enlarges (Θ, S) .

3.3 Membership Inference Attacks

We implement a total of seven attacks that the models can be evaluated against. Below is a description of all seven attacks:

1. *simple_conf_threshold_mia* (given): An attack that reads the logits returned by the predict function, converts it into probabilities using softmax, and returns true if the max probability exceeds a certain threshold.
2. *simple_logits_threshold_mia* (given): An attack that reads the logits returned by the prediction function and returns true if the max value of the logits exceeds a certain threshold.
3. *loss_thresh_mia* (implemented): An attack that computes the cross entropy loss of the prediction logits with the true labels and returns true if the loss is below a certain threshold.
4. *loss_proba_mia* (implemented): An attack that computes the cross entropy loss of the prediction logits with the true labels, compares it with the mean and standard deviation of the training loss, and returns true if the probability of obtaining a loss value as extreme or less extreme than that is lower than a certain threshold. Assumes a normal distribution.
5. *gap_attack_mia* (implemented): An attack that simply returns true if the model made a correct prediction.
6. *augmentation_mia* (implemented): An attack that applies translation and rotation to the input and returns true if the predicted label before and after the transformation remains the same.
7. *shokri_mia* (implemented): An attack that predicts membership from probability vectors using attack models that are trained by reading predictions from shadow models on auxiliary data.

All attacks rely on the model overfitting, which creates a discernible difference between the performance on members and nonmembers. Attacks 3 and 4 analyze the loss of the prediction, where the expected loss on the distribution is higher than the expected loss on the training set [8]. Attacks 5 and 6 are label-only attacks [2] that still work even if the logits returned by the prediction function are obfuscated to only provide the label information. The drawback to this approach is worse accuracy compared to attacks with access to logits or the necessity of several queries. Thresholds and augmentation values were obtained through experimentation and the best-performing values were selected. Attack 7 uses the validation and testing dataset as auxiliary data to train shadow models, whose predictions are used to train a SVM attack model [6]. Table 1 summarizes each attack, their required information, hyperparameter values, accuracy, and advantage for the undefended model.

Attack Name	Requirements/Knowledge	Hyperparameters	Accuracy	Advantage
<i>simple_conf_threshold_mia</i>	logits	thresh=0.999	65.23%	0.305
<i>simple_logits_threshold_mia</i>	logits	thresh=9	65.43%	0.309
<i>loss_thresh_mia</i>	logits, target labels	thresh=0.001	65.62%	0.312
<i>loss_proba_mia</i>	logits, target labels, training loss distribution	thresh=0.402	65.72%	0.314
<i>gap_attack_mia</i>	target labels	None	57.23%	0.145
<i>augmentation_mia</i>	multiple queries	shifts=7, angle=9.9	58.69%	0.174
<i>shokri_mia</i>	target model architecture, auxiliary data	num_shadow=10, num_epochs=15 ¹	58.79%	0.176

Table 1: Performance of membership inference attacks on undefended model

¹See code genshokri.py for full list of parameters

3.4 Adversarial Example Attacks

We evaluate the robustness of the provided target model against four adversarial attacks: one baseline set of examples given in the starter files (Attack0) and three newly implemented attacks. These are FGSM (Fast Gradient Sign Method), PGD (Projected Gradient Descent), and L2 (L2 Attack).

1. *FGSM*[3]: Take one quick step in the direction that increases the model loss the most. Compute the gradient of the loss with respect to the input image. Change each pixel a little bit by adding the sign of that gradient times a small number epsilon. Then clip the result so pixels stay valid.
2. *PGD*[4]: Repeat many small FGSM-like steps and keep the perturbation within a fixed size limit. Start from the original image, then for many iterations compute the loss gradient and take a small step in its sign direction. After each step, clip the perturbation so it stays inside the allowed L-infinity ball. At the end you get a more refined adversarial image.
3. *L2 Attack*[1]: Directly search for a small L2 perturbation that makes the model choose a wrong class, by solving an optimization problem. Instead of using gradient sign steps, it optimizes a continuous perturbation variable with an optimizer like Adam to minimize a loss that trades off two goals: keep the perturbation small in L2 norm, and make the modified image be classified as a different class. The optimization runs for many steps until it finds a good tradeoff.

Attack	Key Parameters	Benign Acc (%)	Adv Acc (%)	Success (%)
Attack0 (given)	–	91.00	6.00	94.00
FGSM	$\epsilon = 8/255$	87.70	22.95	77.05
PGD	$\epsilon = 8/255, \alpha = 2/255, 20$ steps	87.70	0.29	99.71
CW-L2	$c = 1 \times 10^{-2}, 30$ steps	87.70	88.87	11.13

Table 2: Adversarial attack results showing benign accuracy, adversarial accuracy, and success rate for each attack. Success rate is calculated as $100 - \text{Adv Acc}$.

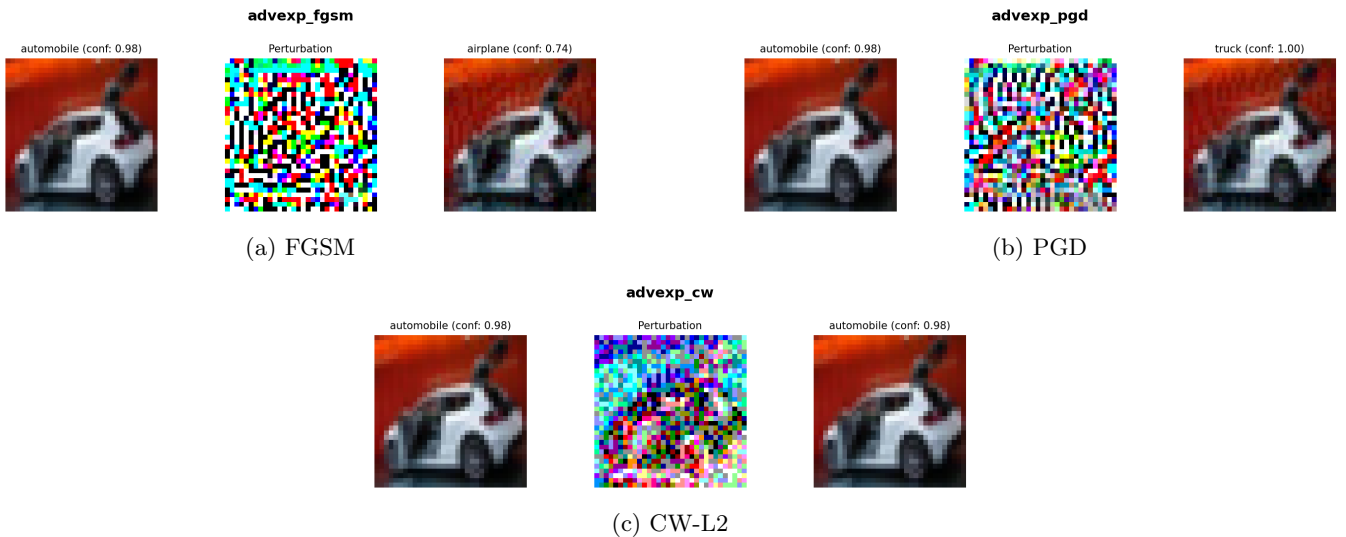


Figure 1: Representative adversarial examples. Each subfigure shows (left) the benign image, (center) the perturbation visualization, and (right) the adversarial image with predicted label and confidence.

Our experiments show that the model performs well on benign images but is sensitive to small adversarial perturbations. FGSM reduced the accuracy from 87.7% to 22.9%, showing that a single-step gradient attack can significantly affect predictions. PGD, which performs multiple iterative steps, was the most effective, dropping accuracy to 0%. However, not all the attacks were successful. The L2 attack caused smaller visual distortions and failed to change the predicted label, leaving accuracy at 88.9%. Overall, these results confirm that while simple gradient-based defenses might slow down weaker attacks, the model remains vulnerable to iterative methods like PGD.

3.5 Part 2: Adversarial Training

For Part 2, we train a ResNet-18 on CIFAR-100 using adversarial training [4]. Unlike Part 1’s prediction-time defense, this approach builds robustness directly into the model during training. We first train with 100% adversarial examples to establish a fully robust baseline, then explore the accuracy-robustness trade-off by training with mixed ratios (25%, 50%, 75% adversarial examples). During each training iteration, we:

1. Generate adversarial examples using PGD ($\epsilon = 8/255$, $\alpha = 2/255$, 7 iterations).
2. Train on either adversarial examples only (100% ratio) or a mixture of clean and adversarial examples (25%, 50%, 75% ratios).

This approach optimizes the model to correctly classify worst-case perturbations within the ℓ_∞ ball. The training configuration uses SGD optimizer (lr=0.01, momentum=0.9, weight decay= 5×10^{-4}) with cosine annealing learning rate scheduler for 20 epochs. We evaluate robustness using PGD attacks at multiple epsilon values and privacy using our MIA suite from Part 1.

The core PGD attack implementation computes perturbations in normalized image space:

```
def pgd_attack(model, images, labels, device, eps=8/255, alpha=2/255, iters=7):
    # Scale eps and alpha to normalized space
    eps_normalized = eps / std
    alpha_normalized = alpha / std

    for i in range(iters):
        adv_images.requires_grad = True
        loss = loss_fn(model(adv_images), labels)
        loss.backward()

        # Update with normalized alpha
        adv_images = adv_images + alpha_normalized * adv_images.grad.sign()
        # Project onto L-inf ball
        eta = torch.clamp(adv_images - original_images,
                          min=-eps_normalized, max=eps_normalized)
        adv_images = torch.clamp(original_images + eta,
                                min=lower_bound, max=upper_bound).detach()

    return adv_images
```

The adversarial training loop generates perturbations on-the-fly:

```
for inputs, targets in trainloader:
    # Generate adversarial examples
    adv_inputs = pgd_attack(model, inputs, targets, device)

    # Train on adversarial examples
    optimizer.zero_grad()
    outputs = model(adv_inputs)
    loss = criterion(outputs, targets)
    loss.backward()
    optimizer.step()
```

Computational Setup. Training was performed on the UF HiPerGator cluster using SLURM. Our job script requested 1 GPU, 4 CPUs, and 32GB RAM with a 4-hour time limit:

```
#SBATCH --gpus=1
#SBATCH --cpus-per-task=4
#SBATCH --mem=32gb
#SBATCH --time=04:00:00
```

The environment was configured using a setup script that installs PyTorch with CUDA 12.1 support and required dependencies (scikit-learn, scipy). Training completed in approximately 44.5 minutes on an NVIDIA L4 GPU.

4 Results

4.1 Part 1 — Defense Performance

Utility–privacy trade-off. We tune only inference-time parameters and intentionally accept a modest drop in clean accuracy to improve privacy and stabilize predictions. The defended model retains strong utility (validation ≈ 0.82 – 0.84) while substantially weakening membership signals: confidence/logit/threshold attacks are driven near random and augmentation-based MIAs are broken by the geometric randomization, whereas loss-distribution MIAs remain comparatively strongest. This trade-off aligns with our design goal and requires no retraining or changes to model weights.

Defended setup (runs=4). Unless otherwise noted, defended results below use $\sigma=0.016$, rotation $\Theta = \pm 5^\circ$, horizontal shift $S=2$ px, temperature $T=1.38$ with jitter $\delta=0.05$, tiny logit noise $\gamma=2\times 10^{-4}$, and EOT averaging with $L=4$ randomized forwards.

Attack Name	Undef. Acc.	Undef. Adv.	Defended Acc.	Defended Adv.
simple_conf_threshold_mia	65.23%	0.305	50.59%	0.012
simple_logits_threshold_mia	65.43%	0.309	50.10%	0.002
loss_thresh_mia	65.62%	0.312	49.80%	-0.004
loss_prob_mia	65.72%	0.314	61.91%	0.238
gap_attack_mia	57.23%	0.145	57.81%	0.156
augmentation_mia	58.69%	0.174	49.71%	-0.006
shokri_mia	58.79%	0.176	60.74%	0.215

Table 3: Membership inference attack performance (accuracy & advantage) before vs. after defense (runs= 4). Lower is better on the right; advantage near 0 indicates near-random performance.

simple_conf_threshold_mia. After defense the attack is nearly random (50.59% / 0.012). Input Gaussian noise and temperature scaling reduce overconfidence and shrink member–non-member separability in the maximum confidence signal, which this attack relies on.

simple_logits_threshold_mia. Also near random (50.10% / 0.002). Softening logits with T and adding tiny logit noise γ blur the sharp margins that a fixed logit threshold exploits.

loss_thresh_mia. Slightly below random (49.80% / -0.004). Small input/logit noises plus temperature scaling make per-example cross-entropy less separable by a fixed threshold.

loss_prob_mia. Remains the strongest among our MIAs (61.91% / 0.238). Tiny temperature jitter and noise reduce, but do not fully flatten, the per-example loss distribution; residual structure enables a probabilistic loss-based discriminator.

gap_attack_mia. Shows a moderate advantage (57.81% / 0.156). Because gap mainly keys on correctness, and we deliberately preserve high clean accuracy, its advantage decreases less than confidence/logit-threshold attacks.

augmentation_mia. Pushed to near random (49.71% / -0.006). Per-forward random rotation/shift explicitly break the fixed-transform consistency assumption, undermining stability-based membership cues.

shokri_mia. Retains a non-trivial advantage (60.74% / 0.215). This class-wise shadow attack exploits the *entire* softmax vector shape; EOT averaging (runs= 4) stabilizes class-specific probability patterns, partially offsetting our small input/logit noises and temperature jitter. If the attack models are trained on defended outputs (domain-matched), the natural domain-shift handicap diminishes, helping the attacker recover separability.

For Adversarial Examples, we also use the same defended setup. We report benign/adv accuracies and define untargeted success as $100\% - (\text{adv. accuracy})$.

Attack	Undef. Success (%)	New Benign Acc (%)	New Adv Acc (%)	New Success (%)
Attack0 (given)	94.00	86.50	49.50	50.50
FGSM	77.05	83.79	30.08	69.92
PGD	99.71	84.47	1.56	98.44
CW-L2	11.13	83.30	83.20	16.80

Table 4: Adversarial example evaluation with the defended predictor (runs= 4). Success is defined as 100% – (adv. accuracy).

Attack0 (given). Benign 86.50%, adversarial 49.50% $\Rightarrow$ success **50.5%** (down from 94%). Input Gaussian smoothing together with EOT averaging reduces the effectiveness of one-shot, small-magnitude perturbations by “averaging out” unstable directions. Tiny per-forward rotations/shifts further desynchronize pixel-aligned artifacts from the model’s features. Overall, Attack0 success drops substantially while clean utility remains acceptable.

FGSM. Benign 83.79%, adversarial 30.08% $\Rightarrow$ success **69.92%** (down from 77.1%). As with Attack0, single-step gradients are partially neutralized by input smoothing and multi-run logit averaging. Geometric randomization ($\pm 5^\circ$, ± 2 px) weakens spatial alignment, making a single forward less exploitable. The improvement is visible but smaller than Attack0 because FGSM uses an explicit gradient step.

PGD. Benign 84.47%, adversarial 1.56% $\Rightarrow$ success **98.44%** (still very high). Iterative optimization adapts to our randomized decision surface across steps, so inference-only randomization yields limited gains against strong multi-step attacks. EOT stabilizes our predictions (good for utility and single-step robustness) but can also smooth away some randomness that would otherwise hinder iterative progress. Substantial PGD mitigation typically requires training-time methods (e.g., adversarial training or certified defenses), which are outside our inference-only scope.

CW-L2. Benign 83.30%, adversarial 83.20% $\Rightarrow$ success **16.8%** (low both before and after defense). This indicates the provided CW configuration is relatively weak or mismatched to our preprocessing/norm bounds (e.g., scale/normalization or targeted vs. untargeted intent). Because the baseline attack already fails to flip most labels, the defense does not materially change its behavior. The takeaway is that our conclusions here are limited by the attack file’s strength rather than by the defense itself.

4.2 Part 2 - Adversarial Training Results

4.2.1 Training Performance

We trained a ResNet-18 on CIFAR-100 with PGD adversarial training for 20 epochs (44.5 minutes on NVIDIA L4 GPU). Table 5 shows the training progression.

Epoch	Train Loss	Train Acc (%)	Val Acc (%)
1	4.549	2.84	7.70
5	3.966	9.60	18.29
10	3.713	13.72	23.88
15	3.547	16.38	30.12
20	3.474	17.61	30.78

Table 5: Part 2 training progress. Train accuracy is measured on adversarial examples; validation accuracy is on clean examples.

The final clean test accuracy is 30.78%. Note that training accuracy is measured on adversarial examples (a harder task), which explains why validation accuracy exceeds training accuracy throughout training.

4.2.2 Robustness Evaluation

Table 6 compares adversarial robustness between the Part 1 defended model (prediction-time defense) and Part 2 (adversarial training).

Model	Clean Acc	PGD ($\epsilon=8/255$)	% Retained
Part 1 defended (CIFAR-10)	84.47%	1.56%	1.8%
Part 2 (adv. trained, CIFAR-100)	30.78%	17.60%	57.2%

Table 6: Robustness comparison. Adversarial training retains 57% of clean accuracy under PGD attack, compared to under 2% for prediction-time defense. Note: datasets differ (CIFAR-10 vs CIFAR-100).

The adversarially trained model retains 57.2% of its clean accuracy under PGD attack ($\epsilon=8/255$), compared to just 1.8% for the Part 1 defended model. Under a stronger attack ($\epsilon=13/255$), the model still retains 36.6% (11.27% accuracy). This demonstrates that adversarial training dramatically improves robustness.

4.2.3 Privacy Evaluation

Table 7 shows MIA results on the adversarially trained model. We adjusted attack thresholds for the 100-class setting (e.g., confidence threshold lowered from 0.999 to 0.4, logits threshold from 9 to 3).

Attack	Part 1 Advantage	Part 2 Advantage	Reduction
Confidence Threshold	0.305	0.018	94.1%
Logits Threshold	0.309	0.027	91.3%
Loss Threshold	0.312	0.006	98.1%
Gap Attack	0.145	-0.014	109.7%
Augmentation	0.174	-0.023	113.2%

Table 7: MIA advantage comparison. Part 2’s adversarially trained model reduces all attack advantages to near-zero.

All MIA attacks achieve near-random accuracy ($\sim 50\%$) with advantages near zero or negative on the adversarially trained model. This represents a 91–113% reduction in attack advantage compared to the Part 1 undefended baseline. The model does not overfit, making training members indistinguishable from non-members.

4.2.4 Key Finding: Robustness-Privacy Synergy

Our most significant finding is that adversarial training provides strong privacy protection as a side effect. This occurs because adversarial training acts as a regularizer, preventing the overfitting that MIA attacks exploit. This suggests that robustness and privacy are complementary rather than conflicting objectives—a model trained to be robust against adversarial examples naturally becomes more private.

4.2.5 Mixed Adversarial Training: Exploring the Trade-off

After establishing that 100% adversarial training provides strong robustness and privacy at the cost of clean accuracy, we investigated whether mixed training ratios could offer a better balance. We trained models with 25%, 50%, and 75% adversarial examples (the remaining samples being clean).

Adv. Ratio	Clean Acc	PGD Acc	% Retained	MIA Adv
25%	49.75%	12.75%	25.6%	0.03
50%	41.82%	16.52%	39.5%	0.01
75%	35.90%	17.19%	47.9%	0.01
100%	30.78%	17.60%	57.2%	0.02

Table 8: Trade-off across different adversarial training ratios. Higher ratios improve robustness retention but reduce clean accuracy. All ratios achieve near-zero MIA advantage.

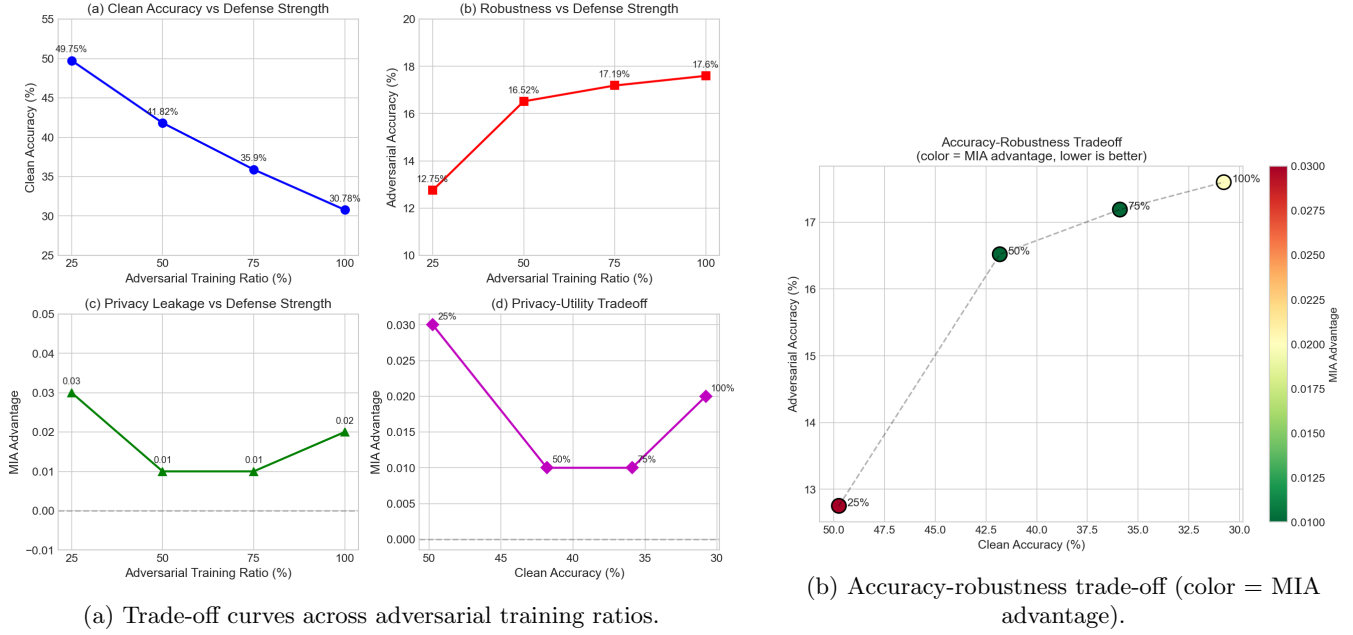


Figure 2: Mixed adversarial training results. (a) Clean accuracy decreases and adversarial accuracy increases with higher adversarial ratios; all ratios achieve near-zero MIA advantage. (b) Moving from 25% to 100% adversarial training increases robustness at the cost of clean accuracy, while maintaining strong privacy.

The results reveal a clear trade-off: increasing the adversarial training ratio improves robustness (from 25.6% retention at 25% ratio to 57.2% at 100%) but decreases clean accuracy (from 49.75% to 30.78%). Importantly, all mixed ratios maintain near-zero MIA advantage (0.01–0.03), indicating that even partial adversarial training provides strong privacy protection. This suggests that practitioners can choose their desired point on the accuracy-robustness curve based on application requirements while maintaining privacy benefits.

5 Conclusions

This project investigated defenses against membership inference and adversarial attacks through two complementary approaches: prediction-time defenses (Part 1) and training-time defenses via adversarial training (Part 2). Our experiments reveal important insights about the relationship between robustness, privacy, and accuracy.

Part 1: No-retraining defense: stronger privacy, constrained robustness. This inference-time defense accepts a modest reduction in clean accuracy (validation ≈ 0.82 – 0.84) in exchange for privacy and stability benefits. Empirically, confidence/logit/threshold MIAs are reduced to near-random levels and augmentation-based MIAs are largely neutralized by small geometric randomization; however, loss-distribution MIAs remain comparatively stronger. Against adversarial examples, single-step attacks show noticeable degradation in effectiveness under our smoothing and EOT averaging, whereas strong iterative attacks (e.g., PGD) remain highly effective. The approach requires no retraining or weight changes but increases inference cost approximately linearly with the number of randomized runs, reflecting an explicit utility–privacy–latency trade-off. And certainly, in order to defend against the attacks that are still working, we need to design some more defense mechanisms.

Part 2: Adversarial training provides strong robustness. The adversarially trained model retains 57% of its clean accuracy under PGD attack ($\epsilon=8/255$), compared to near-zero for the standard model in Part 1. This confirms that training on adversarial examples is an effective approach for building robust models, consistent with prior work [4].

Key Finding: Adversarial training improves privacy. The adversarially trained model achieves near-zero MIA advantage across all attacks, compared to 0.30+ for the Part 1 undefended model. We hypothesize this occurs because adversarial training acts as a strong regularizer, preventing the overfitting that MIAs exploit. Unlike Part 1’s prediction-time defense which only masks overfitting signals, adversarial training fundamentally changes the model to not overfit in the first place. However, we note an important caveat: without a 0% adversarial training baseline on CIFAR-100, we cannot definitively attribute the privacy improvement to adversarial training alone—the difference in dataset complexity (100 classes vs 10) may also contribute to reduced overfitting.

Accuracy-Robustness Trade-offs. The adversarially trained model achieves 30.78% clean accuracy on CIFAR-100 compared to typical 70%+ for standard training. This trade-off is well-documented [7], but the dramatic improvements in both robustness and privacy may justify the accuracy cost for security-critical applications. Our mixed training experiments (25%, 50%, 75% adversarial ratios) show that this trade-off is tunable: 25% adversarial training achieves 49.75% clean accuracy with 25.6% robustness retention, while 100% achieves 30.78% clean accuracy with 57.2% retention. Crucially, all ratios maintain near-zero MIA advantage, indicating that even partial adversarial training provides strong privacy benefits.

Comparing the Two Approaches. Part 1’s prediction-time defense is attractive because it requires no re-training, but its protection is limited and can potentially be circumvented by adaptive attacks. Part 2’s adversarial training provides stronger guarantees for both robustness and privacy, but at the cost of reduced clean accuracy and increased training time. The choice between approaches depends on the application’s requirements and constraints.

Limitations and Future Work. Our Part 1 defense is limited to prediction-time randomization: it drives most confidence/augmentation MIAs near random and helps single-step AEs, but loss-based MIAs remain comparatively strong, PGD remains effective, and latency grows with runs. Next we will: (i) apply *training-time* L2 weight-decay fine-tuning (no architecture change) to reduce overfitting and specifically weaken Shokri’s class-wise shadow attack; (ii) evaluate *adaptive* attackers, including domain-matched Shokri (trained on defended outputs) and EOT-aware PGD; (iii) harden the output space at inference via top- k probability release, probability quantization (2–3 decimals), and small uniform mixing, plus run scheduling (smaller runs for MIA, larger for AE) to balance privacy–utility–latency; and (iv) try adversarial training to defense adversarial examples.

Our Part 2 model was trained for only 20 epochs; longer training would likely improve both clean and adversarial accuracy. We did not evaluate against adaptive attacks specifically designed to bypass our defenses.

Transferability and Baseline Limitations. A key limitation is that our Part 1 and Part 2 experiments use different datasets (CIFAR-10 vs CIFAR-100), making direct comparisons difficult. More importantly, we did not train a 0% adversarial baseline on CIFAR-100, so we cannot isolate whether the observed privacy improvements stem from adversarial training itself or from the inherent properties of the harder 100-class task (which may naturally reduce overfitting). Future work should train standard models on CIFAR-100 to establish a proper baseline and confirm whether the robustness-privacy synergy transfers across datasets.

Future work could explore combining adversarial training with differential privacy for formal privacy guarantees, or investigate whether the robustness-privacy synergy holds for other architectures and datasets. Additionally, training with a deeper model for more epochs may give better accuracy and utility to the implementation.

References

- [1] Nicholas Carlini and David Wagner. Towards evaluating the robustness of neural networks, 2017.
- [2] Christopher A. Choquette-Choo, Florian Tramèr, Nicholas Carlini, and Nicolas Papernot. Label-only membership inference attacks. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, pages 1964–1974. PMLR, 2021.
- [3] Ian J. Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial examples, 2015.
- [4] Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, and Adrian Vladu. Towards deep learning models resistant to adversarial attacks. *arXiv preprint arXiv:1706.06083*, 2017.
- [5] Ahmed Salem, Yang Zhang, Mathias Humbert, Pascal Berrang, Mario Fritz, and Michael Backes. MI-leaks: Model and data independent membership inference attacks and defenses on machine learning models. *arXiv preprint*, arXiv:1806.01246, 2018. [cs.CR].
- [6] Reza Shokri, Marco Stronati, Congzheng Song, and Vitaly Shmatikov. Membership inference attacks against machine learning models. In *2017 IEEE Symposium on Security and Privacy (SP)*, pages 3–18. IEEE, 2017.
- [7] Dimitris Tsipras, Shibani Santurkar, Logan Engstrom, Alexander Turner, and Aleksander Madry. Robustness may be at odds with accuracy. In *International Conference on Learning Representations*, 2019.
- [8] Samuel Yeom, Irene Giacomelli, Matthew Fredrikson, and Somesh Jha. Privacy risk in machine learning: Analyzing the connection to overfitting. In *2018 IEEE 31st Computer Security Foundations Symposium (CSF)*, pages 268–282. IEEE, 2018.